

```

1  import logging
2  import os
3  import time
4
5  import pandas as pd
6  import yfinance as yf
7
8  from utils import ls_t1101, ls_t2101, ls_g3101
9  from utils.product_master import get_product
10
11 CSV_FILE = "data/trade_history.csv"
12 _NAME_CACHE = {} # 종목명 캐싱용 딕셔너리
13 # LS t1101 응답 캐시 (동일 종목 연속 조회·Streamlit 재실행 시 과도한 API 방지)
14 _LS_T1101_CACHE = {} # shcode -> (monotonic_ts, outblock dict | None)
15 _LS_T1101_CACHE_TTL_SEC = 60.0
16 _LS_G3101_CACHE = {} # symbol -> (ts, data)
17 _LS_G3101_CACHE_TTL_SEC = 60.0
18
19 logger = logging.getLogger(__name__)
20
21 # 노출통화: 환율(USD/KRW) 리스크 기준 - 거래 결제 통화(currency)와 별개
22 EXPOSURE_CURRENCY_OPTIONS = ("KRW", "USD")
23 # 자산군 (향후개발계획 1.2)
24 ASSET_CLASS_OPTIONS = ("지수형", "개별주식", "채권형", "선물")
25 # 기존 매매 마이그레이션 시 기본값 (요청사항)
26 LEGACY_DEFAULT_EXPOSURE_CURRENCY = "KRW"
27 LEGACY_DEFAULT_ASSET_CLASS = "지수형"
28 # 신규 매매 입력 시 자산군 기본값
29 NEW_TRADE_DEFAULT_ASSET_CLASS = "개별주식"
30
31
32 def _normalize_exposure_currency(val):
33     if val is None or pd.isna(val):
34         return LEGACY_DEFAULT_EXPOSURE_CURRENCY
35     s = str(val).strip().upper()
36     return "USD" if s == "USD" else "KRW"
37
38
39 def _normalize_asset_class(val):
40     if val is None or pd.isna(val):
41         return LEGACY_DEFAULT_ASSET_CLASS
42     s = str(val).strip()
43     return s if s in ASSET_CLASS_OPTIONS else LEGACY_DEFAULT_ASSET_CLASS
44
45
46 def _apply_futures_trade(q_old, cost_basis_krw, dq, price, multiplier=1.0):
47     """
48     선물: 매수 +dq(롱), 매도 -dq(숏). 양방향 포지션 가능.
49     수량은 이미 multiplier가 반영된 상태.
50     비용 계산: (거래량 × 가격)
51     Returns: (new_quantity, new_cost_krw, realized_pnl_krw)
52     """
53     # 입력된 수량에 승수를 곱해서 실제 수량으로 변환
54     dq_actual = dq * multiplier
55     realized = 0.0
56
57     if abs(q_old) < 1e-12:
58         # 포지션이 없던 상태 → 새로 오픈
59         return dq_actual, dq_actual * price, realized
60
61     avg = cost_basis_krw / q_old
62
63     if q_old * dq_actual > 0:
64         # 같은 방향 추가 (롱에 롱 추가, 숏에 숏 추가)
65         return q_old + dq_actual, cost_basis_krw + dq_actual * price, realized
66
67     # 반대 방향 거래 (기존 포지션을 청산하는 경우)
68     c = min(abs(q_old), abs(dq_actual))
69     if q_old > 0:
70         # 롱 포지션 청산 (매도)
71         realized += (price - avg) * c
72     else:

```

```

73         # 숏 포지션 청산 (매수로 청산)
74         realized += (avg - price) * c
75
76     q_new = q_old + dq_actual
77     if abs(q_new) < 1e-12:
78         # 포지션 전부 청산
79         return 0.0, 0.0, realized
80
81     if (q_new > 0) == (q_old > 0):
82         # 같은 방향으로 남음 (롱 또는 숏)
83         return q_new, avg * q_new, realized
84
85     # 반대 방향 포지션으로 전환
86     return q_new, q_new * price, realized
87
88
89 def _is_future_ticker(ticker):
90     """선물 여부 판별 (8자리 코드 or product_master에서 asset_class='선물')"""
91     ticker_str = str(ticker).strip().upper()
92     product_info = get_product(ticker_str)
93     return (product_info and product_info.get("asset_class") == "선물") or len(
94         ticker_str
95     ) == 8
96
97
98 def load_trade_history():
99     if not os.path.exists(CSV_FILE):
100         return pd.DataFrame(
101             columns=[
102                 "date",
103                 "account",
104                 "ticker",
105                 "tradeType",
106                 "quantity",
107                 "price",
108                 "currency",
109                 "fx_krw_per_usd",
110                 "exposure_currency",
111                 "asset_class",
112             ]
113         )
114
115     df = pd.read_csv(CSV_FILE)
116
117     changed = False
118     # 기존 데이터에 currency 컬럼이 없는 경우 추가 (기본값 USD)
119     if "currency" not in df.columns:
120         df["currency"] = "USD"
121         changed = True
122
123     # 기존 데이터에 account 컬럼이 없는 경우 추가 (기본값 기본계좌)
124     if "account" not in df.columns:
125         df["account"] = "기본계좌"
126         changed = True
127
128     # 매매 시점 USD/KRW 환율 (1 USD당 원화). KRW 거래는 1.0
129     if "fx_krw_per_usd" not in df.columns:
130         df["fx_krw_per_usd"] = pd.NA
131         df.loc[df["currency"] == "KRW", "fx_krw_per_usd"] = 1.0
132         changed = True
133
134     if "exposure_currency" not in df.columns:
135         df["exposure_currency"] = LEGACY_DEFAULT_EXPOSURE_CURRENCY
136         changed = True
137     if "asset_class" not in df.columns:
138         df["asset_class"] = LEGACY_DEFAULT_ASSET_CLASS
139         changed = True
140
141     if "exposure_currency" in df.columns and df["exposure_currency"].isna().any():
142         df["exposure_currency"] = df["exposure_currency"].fillna(
143             LEGACY_DEFAULT_EXPOSURE_CURRENCY
144         )

```

```

145         changed = True
146     if "asset_class" in df.columns and df["asset_class"].isna().any():
147         df["asset_class"] = df["asset_class"].fillna(LEGACY_DEFAULT_ASSET_CLASS)
148         changed = True
149
150     if changed:
151         df.to_csv(CSV_FILE, index=False)
152
153     return df
154
155
156 def _normalize_fx_for_row(currency, fx_krw_per_usd, spot_fallback):
157     """저장/표시용 환율. KRW는 1, USD는 입력값(없으면 조회 시점 환율로 대체)."""
158     if currency == "KRW":
159         return 1.0
160     if fx_krw_per_usd is None or pd.isna(fx_krw_per_usd):
161         return float(spot_fallback)
162     return float(fx_krw_per_usd)
163
164
165 def add_trade(
166     date,
167     account,
168     ticker,
169     tradeType,
170     quantity,
171     price,
172     currency="KRW",
173     fx_krw_per_usd=None,
174     exposure_currency=None,
175     asset_class=None,
176 ):
177     df = load_trade_history()
178     spot = get_exchange_rate()
179     fx = _normalize_fx_for_row(currency, fx_krw_per_usd, spot)
180     exp = _normalize_exposure_currency(exposure_currency)
181     ac = _normalize_asset_class(asset_class or NEW_TRADE_DEFAULT_ASSET_CLASS)
182     new_row = pd.DataFrame(
183         [
184             {
185                 "date": date,
186                 "account": account,
187                 "ticker": ticker.upper(),
188                 "tradeType": tradeType,
189                 "quantity": float(quantity),
190                 "price": float(price),
191                 "currency": currency,
192                 "fx_krw_per_usd": fx,
193                 "exposure_currency": exp,
194                 "asset_class": ac,
195             }
196         ]
197     )
198     df = pd.concat([df, new_row], ignore_index=True)
199     df.to_csv(CSV_FILE, index=False)
200
201
202 def update_trade(
203     index,
204     date,
205     account,
206     ticker,
207     tradeType,
208     quantity,
209     price,
210     currency,
211     fx_krw_per_usd=None,
212     exposure_currency=None,
213     asset_class=None,
214 ):
215     df = load_trade_history()
216     if 0 <= index < len(df):

```

```

217     spot = get_exchange_rate()
218     fx = _normalize_fx_for_row(currency, fx_krw_per_usd, spot)
219     exp = _normalize_exposure_currency(exposure_currency)
220     ac = _normalize_asset_class(asset_class)
221     df.loc[index, "date"] = date
222     df.loc[index, "account"] = account
223     df.loc[index, "ticker"] = ticker.upper()
224     df.loc[index, "tradeType"] = tradeType
225     df.loc[index, "quantity"] = float(quantity)
226     df.loc[index, "price"] = float(price)
227     df.loc[index, "currency"] = currency
228     df.loc[index, "fx_krw_per_usd"] = fx
229     df.loc[index, "exposure_currency"] = exp
230     df.loc[index, "asset_class"] = ac
231     df.to_csv(CSV_FILE, index=False)
232     return True
233 return False
234
235
236 def delete_trade(index):
237     df = load_trade_history()
238     if 0 <= index < len(df):
239         df = df.drop(index)
240         df.to_csv(CSV_FILE, index=False)
241         return True
242     return False
243
244
245 def _ls_shcode_from_ticker(ticker):
246     """LS증권 6자리 종목코드. 국내 주식은 정수 문자열, ETF/ETN 등은 영숫자 혼합 6자리."""
247     t = str(ticker).strip().upper()
248     # 6자리 영숫자 (ETF 등) 또는 1~6자리 숫자 (일반 주식)
249     if t.isalnum() and len(t) == 6:
250         return t
251     if t.isdigit() and 1 <= len(t) < 6:
252         return t.zfill(6)
253     return None
254
255
256 def _ls_price_to_float(outblock):
257     if not outblock:
258         return 0.0
259     p = outblock.get("price")
260     if p is None or p == "":
261         return 0.0
262     try:
263         return float(str(p).replace(",", "", "").strip())
264     except ValueError:
265         return 0.0
266
267
268 def _ls_previous_close_to_float(outblock):
269     """t1101 응답에서 전일 종가(jnilclose) 추출"""
270     if not outblock:
271         return 0.0
272     p = outblock.get("jnilclose")
273     if p is None or p == "":
274         return 0.0
275     try:
276         return float(str(p).replace(",", "", "").strip())
277     except ValueError:
278         return 0.0
279
280
281 def _get_ls_t1101_cached(shcode):
282     """t1101 호출 결과 캐시(짧은 TTL). 가격·종목명 조회를 한 번으로 묶기 위함.
283
284     get_current_with_fallback()을 사용하여 자동 재시도 지원:
285     1차 시도: shcode 그대로
286     2차 시도: shcode.KS (일부 ETF 등이 필요)
287     """
288     now = time.monotonic()

```

```

289     if shcode in _LS_T1101_CACHE:
290         ts, data = _LS_T1101_CACHE[shcode]
291         if now - ts < _LS_T1101_CACHE_TTL_SEC:
292             return data
293
294     data = ls_t1101.get_current_with_fallback(shcode)
295     _LS_T1101_CACHE[shcode] = (now, data)
296     return data
297
298
299 _LS_T2101_CACHE = {} # shcode -> (monotonic_ts, outblock dict | None)
300
301
302 def _get_ls_g3101_cached(symbol):
303     now = time.monotonic()
304     if symbol in _LS_G3101_CACHE:
305         ts, data = _LS_G3101_CACHE[symbol]
306         if now - ts < _LS_G3101_CACHE_TTL_SEC:
307             return data
308     data = ls_g3101.get_current(symbol)
309     _LS_G3101_CACHE[symbol] = (now, data)
310     return data
311
312
313 def _get_ls_t2101_cached(shcode):
314     """
315     t2111 선물 API 캐시
316     """
317     now = time.monotonic()
318     if shcode in _LS_T2101_CACHE:
319         ts, data = _LS_T2101_CACHE[shcode]
320         if now - ts < _LS_T1101_CACHE_TTL_SEC:
321             return data
322
323     # t2111로 직접 조회
324     data = ls_t2101.get_future_current_price(shcode)
325     _LS_T2101_CACHE[shcode] = (now, data)
326     return data
327
328
329 def get_current_price(ticker):
330     """현재가 조회. product_master 기반 데이터 소스 선택
331
332     - market='KRX', asset_class='선물': LS t2101 (국내 선물)
333     - market='KRX', asset_class='개별주식': LS t1101 (국내 주식)
334     - market='NYSE'/'NASDAQ', asset_class='개별주식': LS g3101 (미국 주식)
335     - 기타: yfinance 폴백
336     """
337
338     ticker_str = str(ticker).strip().upper()
339     product_info = get_product(ticker_str)
340
341     # product_master에서 정보 추출
342     if product_info:
343         market = product_info.get("market", "").upper()
344         asset_class = product_info.get("asset_class", "")
345
346         # 1. 국내 선물 (KRX + 선물)
347         if market == "KRX":
348             if asset_class == "선물":
349                 ob = _get_ls_t2101_cached(ticker_str)
350                 price = _ls_price_to_float(ob)
351                 if price >= 0:
352                     return price
353             else: # 2. 국내 주식 (KRX + 개별주식)
354                 shcode = _ls_shcode_from_ticker(ticker_str)
355                 if shcode:
356                     ob = _get_ls_t1101_cached(shcode)
357                     price = _ls_price_to_float(ob)
358                     if price >= 0:
359                         return price
360

```

```

361         # 3. 미국 주식 (NYSE/NASDAQ + 개별주식)
362         elif market in ("NYSE", "NASDAQ"):
363             try:
364                 price = ls_g3101.get_current(ticker_str, exchange=market)
365                 if price and price >= 0:
366                     return price
367             except Exception as e:
368                 logger.warning("LS g3101 error for %s (%s): %s", ticker_str, market, e
369
370
371         # yfinance로 풀백하지 않고 0으로 리턴하며 warning 로그를 남김
372         logger.warning("No valid price source for %s, returning 0.0", ticker_str)
373         return 0.0
374
375 def get_previous_close_price(ticker):
376     """전일 종가 반환. 국내: LS API → .KS 재시도 → 해외(yfinance)"""
377
378     ticker_str = str(ticker).strip().upper()
379     product_info = get_product(ticker_str)
380     is_future = (product_info and product_info.get("asset_class") == "선물") or len(
381         ticker_str
382     ) == 8
383
384     if is_future:
385         ob = _get_ls_t2101_cached(ticker_str)
386         price = _ls_previous_close_to_float(ob)
387         if price > 0:
388             return price
389
390     shcode = _ls_shcode_from_ticker(ticker_str)
391     if shcode:
392         ob = _get_ls_t1101_cached(shcode)
393         price = _ls_previous_close_to_float(ob)
394         if price > 0: # 성공
395             return price
396         # LS API 실패 → 해외주식으로 처리
397
398     # yfinance로 해외주식 전일 종가 조회
399     try:
400         stock = yf.Ticker(ticker_str)
401         hist = stock.history(period="5d") # 5일 데이터로 전일자 가져오기
402         if len(hist) >= 2:
403             return float(hist["Close"].iloc[-2]) # 최신 바로 이전이 전일자
404         return 0.0
405     except Exception as e:
406         logger.warning("Error fetching previous close for %s: %s", ticker_str, e)
407         return 0.0
408
409
410 def get_company_name(ticker):
411     """종목 코드로 회사 이름. product_master → 국내: LS API → .KS 재시도 → 해외(yfinance) → 코드
412     반환"""
413
414     if ticker in _NAME_CACHE:
415         return _NAME_CACHE[ticker]
416
417     ticker_str = str(ticker).strip().upper()
418     product_info = get_product(ticker_str)
419
420     # 1. product_master.json에서 name 필드 확인 (최우선)
421     if product_info and product_info.get("name"):
422         name = str(product_info["name"]).strip()
423         if name:
424             _NAME_CACHE[ticker] = name
425         return name
426
427     is_future = (product_info and product_info.get("asset_class") == "선물") or len(
428         ticker_str
429     ) == 8
430
431     if is_future:

```

```

431     ob = _get_ls_t2101_cached(ticker_str)
432     if ob and ob.get("hname"):
433         name = str(ob["hname"]).strip()
434         _NAME_CACHE[ticker] = name
435         return name
436
437 shcode = _ls_shcode_from_ticker(ticker_str)
438 if shcode:
439     ob = _get_ls_t1101_cached(shcode)
440     if ob and ob.get("hname"):
441         name = str(ob["hname"]).strip()
442         _NAME_CACHE[ticker] = name
443         return name
444     # LS API 실패 → 해외주식으로 처리
445
446 try:
447     stock = yf.Ticker(ticker_str)
448     info = stock.info
449     name = info.get("shortName") or info.get("longName") or ticker_str
450 except Exception:
451     name = ticker_str
452
453 _NAME_CACHE[ticker] = name
454 return name
455
456
457 def get_exchange_rate():
458     try:
459         # USD/KRW 환율 가져오기
460         rate = yf.Ticker("USDKRW=X").history(period="1d")
461         if not rate.empty:
462             return rate["Close"].iloc[0]
463         return 1300.0 # API 실패 시 임시 기본값
464     except:
465         return 1300.0
466
467
468 def calculate_portfolio(account_filter=None, tag_filter=None):
469     df = load_trade_history()
470     if df.empty:
471         return pd.DataFrame()
472
473     # 계좌 필터링 적용
474     if account_filter and account_filter != "전체 계좌":
475         df = df[df["account"] == account_filter]
476
477     # 태그 필터링 적용
478     if tag_filter and tag_filter != "전체 태그":
479         # product_master에서 해당 태그를 가진 종목들의 티커를 가져옵니다.
480         from utils.product_master import load_product_master
481
482         products = load_product_master()
483         tickers_with_tag = [
484             t
485             for t, info in products.items()
486             if "tags" in info and tag_filter in info["tags"]
487         ]
488         df = df[df["ticker"].isin(tickers_with_tag)]
489
490     if df.empty:
491         return pd.DataFrame()
492
493     df = df.copy()
494     df["date"] = pd.to_datetime(df["date"], errors="coerce")
495     df = df.sort_values("date", kind="mergesort")
496
497     spot = get_exchange_rate()
498     portfolio = {}
499
500     for _, row in df.iterrows():
501         ticker = row["ticker"]
502         tradeType = row["tradeType"]

```

```

503     quantity = float(row["quantity"])
504     price = float(row["price"])
505     currency = row["currency"]
506     fx = _normalize_fx_for_row(currency, row.get("fx_krw_per_usd"), spot)
507     exp = _normalize_exposure_currency(row.get("exposure_currency"))
508     ac = _normalize_asset_class(row.get("asset_class"))
509
510     if ticker not in portfolio:
511         portfolio[ticker] = {
512             "currentQuantity": 0.0,
513             "totalCostKrw": 0.0,
514             "totalCostUsd": 0.0,
515             "averageCost": 0.0,
516             "averageCostKrw": 0.0,
517             "realizedPnlKrw": 0.0,
518             "currency": currency,
519             "exposure_currency": exp,
520             "asset_class": ac,
521             "companyName": get_company_name(ticker),
522             "tags": get_product(ticker).get("tags", []),
523         }
524     else:
525         portfolio[ticker]["exposure_currency"] = exp
526         portfolio[ticker]["asset_class"] = ac
527
528     # 선물 처리 (양방향 포지션 가능)
529     if _is_future_ticker(ticker):
530         dq = quantity if tradeType in ["Buy", "매수"] else -quantity
531         q_old = portfolio[ticker]["currentQuantity"]
532         cost_old = portfolio[ticker]["totalCostKrw"]
533         product_info = get_product(ticker)
534         multiplier = (
535             float(product_info.get("multiplier", 1.0)) if product_info else 1.0
536         )
537
538         qn, cn, r = _apply_futures_trade(q_old, cost_old, dq, price, multiplier)
539         portfolio[ticker]["currentQuantity"] = qn
540         portfolio[ticker]["totalCostKrw"] = cn
541         portfolio[ticker]["totalCostUsd"] = 0.0
542         portfolio[ticker]["realizedPnlKrw"] += r
543         if abs(qn) > 1e-12:
544             portfolio[ticker]["averageCost"] = cn / qn
545             portfolio[ticker]["averageCostKrw"] = cn / qn
546         else:
547             portfolio[ticker]["averageCost"] = 0.0
548             portfolio[ticker]["averageCostKrw"] = 0.0
549         continue
550
551     # 매수 처리 (한글/영문 모두 지원) - 선물 제외
552     if tradeType in ["Buy", "매수"]:
553         portfolio[ticker]["currentQuantity"] += quantity
554
555         if currency == "KRW":
556             portfolio[ticker]["totalCostKrw"] += quantity * price
557             q = portfolio[ticker]["currentQuantity"]
558             portfolio[ticker]["averageCost"] = portfolio[ticker]["totalCostKrw"] /
559                 q
560             portfolio[ticker]["averageCostKrw"] = portfolio[ticker]["averageCost"]
561         else:
562             portfolio[ticker]["totalCostUsd"] += quantity * price
563             portfolio[ticker]["totalCostKrw"] += quantity * price * fx
564             q = portfolio[ticker]["currentQuantity"]
565             portfolio[ticker]["averageCost"] = portfolio[ticker]["totalCostUsd"] /
566                 q
567             portfolio[ticker]["averageCostKrw"] = (
568                 portfolio[ticker]["totalCostKrw"] / q
569             )
570
571     # 매도 처리 (한글/영문 모두 지원) - 선물 제외
572     elif tradeType in ["Sell", "매도"]:
573         if portfolio[ticker]["currentQuantity"] < quantity:
574             continue

```



```

573
574     if currency == "KRW":
575         avg_krw = portfolio[ticker]["averageCostKrw"]
576         portfolio[ticker]["realizedPnlKrw"] += (price - avg_krw) * quantity
577         portfolio[ticker]["totalCostKrw"] -= avg_krw * quantity
578         portfolio[ticker]["currentQuantity"] -= quantity
579         if portfolio[ticker]["currentQuantity"] == 0:
580             portfolio[ticker]["averageCost"] = 0.0
581             portfolio[ticker]["averageCostKrw"] = 0.0
582             portfolio[ticker]["totalCostKrw"] = 0.0
583         else:
584             portfolio[ticker]["averageCost"] = (
585                 portfolio[ticker]["totalCostKrw"]
586                 / portfolio[ticker]["currentQuantity"]
587             )
588             portfolio[ticker]["averageCostKrw"] = portfolio[ticker][
589                 "averageCost"
590             ]
591     else:
592         avg_usd = portfolio[ticker]["averageCost"]
593         avg_krw = portfolio[ticker]["averageCostKrw"]
594         proceeds_krw = fx * price * quantity
595         cost_krw = avg_krw * quantity
596         portfolio[ticker]["realizedPnlKrw"] += proceeds_krw - cost_krw
597         portfolio[ticker]["totalCostUsd"] -= avg_usd * quantity
598         portfolio[ticker]["totalCostKrw"] -= cost_krw
599         portfolio[ticker]["currentQuantity"] -= quantity
600         if portfolio[ticker]["currentQuantity"] == 0:
601             portfolio[ticker]["averageCost"] = 0.0
602             portfolio[ticker]["averageCostKrw"] = 0.0
603             portfolio[ticker]["totalCostUsd"] = 0.0
604             portfolio[ticker]["totalCostKrw"] = 0.0
605         else:
606             q = portfolio[ticker]["currentQuantity"]
607             portfolio[ticker]["averageCost"] = (
608                 portfolio[ticker]["totalCostUsd"] / q
609             )
610             portfolio[ticker]["averageCostKrw"] = (
611                 portfolio[ticker]["totalCostKrw"] / q
612             )
613
614 # 보유 수량이 있거나 실현 손익이 있는 종목만 필터링
615 active_portfolio = {
616     k: v
617     for k, v in portfolio.items()
618     if abs(v["currentQuantity"]) > 1e-12 or v["realizedPnlKrw"] != 0
619 }
620
621 if not active_portfolio:
622     return pd.DataFrame()
623
624 result_df = pd.DataFrame.from_dict(active_portfolio, orient="index").reset_index()
625 result_df.rename(columns={"index": "ticker"}, inplace=True)
626
627 # 현재가 가져오기 (보유 수량이 있는 경우만)
628 def get_price_if_active(row):
629     if abs(row["currentQuantity"]) > 1e-12:
630         return get_current_price(row["ticker"])
631     return 0.0
632
633 result_df["currentPrice"] = result_df.apply(get_price_if_active, axis=1)
634
635 # 평가 금액(표시 통화) 및 원화 기준 지표
636 def row_current_value_local(r):
637     return r["currentQuantity"] * r["currentPrice"]
638
639 result_df["currentValue"] = result_df.apply(row_current_value_local, axis=1)
640
641 def row_current_value_krw(r):
642     if r["asset_class"] == "선물":
643         return 0.0
644     if r["currency"] == "USD":

```

```

645         return r["currentQuantity"] * r["currentPrice"] * spot
646     return r["currentValue"]
647
648 result_df["currentValueKrw"] = result_df.apply(row_current_value_krw, axis=1)
649
650 def row_unrealized_local(r):
651     if abs(r["currentQuantity"]) < 1e-12:
652         return 0.0
653     return (r["currentPrice"] - r["averageCost"]) * r["currentQuantity"]
654
655 result_df["unrealizedPnl"] = result_df.apply(row_unrealized_local, axis=1)
656
657 def row_unrealized_krw(r):
658     if abs(r["currentQuantity"]) < 1e-12:
659         return 0.0
660
661     if r["asset_class"] == "선물":
662         # 선물: 수량이 이미 승수가 반영됨
663         return (r["currentPrice"] - r["averageCostKrw"]) * r["currentQuantity"]
664
665     if r["currency"] == "USD":
666         return r["currentValueKrw"] - r["totalCostKrw"]
667
668     return r["unrealizedPnl"]
669
670 result_df["unrealizedPnlKrw"] = result_df.apply(row_unrealized_krw, axis=1)
671
672 result_df["realizedPnlKrw"] = result_df["realizedPnlKrw"].astype(float)
673
674 def row_return_local(r):
675     if abs(r["currentQuantity"]) < 1e-12 or abs(r["averageCost"]) < 1e-12:
676         return 0.0
677     if r["asset_class"] == "선물":
678         return 0.0
679     return (r["currentPrice"] - r["averageCost"]) / r["averageCost"] * 100
680
681 result_df["returnRate"] = result_df.apply(row_return_local, axis=1)
682
683 def row_return_krw(r):
684     if abs(r["currentQuantity"]) < 1e-12:
685         return 0.0
686     if r["asset_class"] == "선물":
687         return 0.0
688     base = r["totalCostKrw"]
689     if base <= 0:
690         return 0.0
691     return r["unrealizedPnlKrw"] / base * 100
692
693 result_df["returnRateKrw"] = result_df.apply(row_return_krw, axis=1)
694
695 # 원래 통화 모드용 실현손익: KRW 종목은 원화, USD 종목은 대략적인 USD(현재 환율로 환산)
696 def row_realized_local(r):
697     if r["currency"] == "KRW":
698         return float(r["realizedPnlKrw"])
699     return float(r["realizedPnlKrw"]) / spot if spot else 0.0
700
701 result_df["realizedPnl"] = result_df.apply(row_realized_local, axis=1)
702
703 # 전일 평가손익 계산 (현재 보유 수량 기준)
704 def row_prev_close_price(r):
705     return get_previous_close_price(r["ticker"])
706
707 result_df["previousClosePrice"] = result_df.apply(row_prev_close_price, axis=1)
708
709 # 전일 평가손익 (원화 기준) - 현재 보유 수량으로 전일 가격 계산
710 def row_prev_unrealized_krw(r):
711     if abs(r["currentQuantity"]) < 1e-12:
712         return 0.0
713
714     if r["asset_class"] == "선물":
715         # 선물: 수량이 이미 승수가 반영됨
716         return (r["previousClosePrice"] - r["averageCostKrw"]) * r[

```

```

717         "currentQuantity"
718     ]
719
720     if r["currency"] == "USD":
721         prev_value_krw = r["previousClosePrice"] * spot
722         return (prev_value_krw - r["averageCostKrw"]) * r["currentQuantity"]
723     else:
724         return (r["previousClosePrice"] - r["averageCost"]) * r["currentQuantity"]
725
726 result_df["prevUnrealizedPnlKrw"] = result_df.apply(row_prev_unrealized_krw, axis=
1)
727
728 # 전일대비 평가손익 변동 (오늘 - 어제)
729 def row_pnl_change_krw(r):
730     return r["unrealizedPnlKrw"] - r["prevUnrealizedPnlKrw"]
731
732 result_df["pnlChangeKrw"] = result_df.apply(row_pnl_change_krw, axis=1)
733
734 # 전일대비 변동률 (%) - 가격 변동률 기준: (현재가 / 전일가 - 1) * 100
735 def row_pnl_change_rate(r):
736     if abs(r["previousClosePrice"]) < 1e-12:
737         return 0.0
738     return ((r["currentPrice"] / r["previousClosePrice"]) - 1.0) * 100
739
740 result_df["pnlChangeRate"] = result_df.apply(row_pnl_change_rate, axis=1)
741
742 # USDKRW(선물·헤지): 평가 「금액」 만 0으로 두어 총자산·비중·차트에 반영하지 않음.
743 # 평가손익은 MTM 그대로, 수익률은 의미 없어 0% 표시.
744
745 # 선물 포지션도 평가 「금액」 만 0으로 (명목 가치를 자산에서 제외)
746 _futures = result_df["asset_class"] == "선물"
747 result_df.loc[_futures, "currentValue"] = 0.0
748 result_df.loc[_futures, "currentValueKrw"] = 0.0
749 result_df.loc[_futures, "returnRate"] = 0.0
750 result_df.loc[_futures, "returnRateKrw"] = 0.0
751
752 result_df = result_df.drop(
753     columns=["totalCostKrw", "totalCostUsd"], errors="ignore"
754 )
755
756 return result_df
757

```